STA 336: Statistical Machine Learning

Homework 6

Rento Saijo

April 1, 2026

Table of Contents

Disclosure **2**

Problem 1 **3**

Problem 2 **5**
- Problem 2 (a) . . . 6
- Problem 2 (b) . . . 8
- Problem 2 (c) . . . 9
- Problem 2 (d) . . . 11
- Problem 2 (e) . . . 12
- Problem 2 (f) . . . 13

Problem 3 **15**
- Problem 3 (a) . . . 16
- Problem 3 (b) . . . 17
- Problem 3 (c) . . . 18

Disclosure

GPT-5.4 Codex was used to create the `YAML` portion and some `LaTeX` code to format the text/equations nicely. Page formatting code was also provided by Derin Gezgin. In the setup chunk, libraries were loaded and helper functions such as `table_latex()` and `predict.regsubsets()` were defined to keep the analysis reproducible and readable. See the original `RMD` file here for more details.

Problem 1

Problem 1

In our linear model selection lab, we used an indirect method for choosing the optimal model. Following the validation-set approach from the textbook, use the `Auto` data from `ISLR2`, split the data into training and test sets, compare the candidate models by test-set mean squared error, and select a final model.

Cell 1

```r
# Load data.
auto_df <- ISLR2::Auto

# Split data as shown in textbook (a little different to our typical convention).
set.seed(1)
auto_train <- sample(c(TRUE, FALSE), nrow(auto_df), replace = TRUE)
auto_test  <- !auto_train

# Fit best subset models on training data.
auto_fit_train <- leaps::regsubsets(
  mpg ~ . - name,
  data  = auto_df[auto_train, ],
  nvmax = 7
)

# Compute test-set MSE for each model size.
auto_test_mse <- rep(NA_real_, 7)
for (i in 1:7) {
  auto_pred        <- predict(auto_fit_train, auto_df[auto_test, ], id = i)
  auto_test_mse[i] <- mean((auto_df$mpg[auto_test] - auto_pred)^2)
}

# Create summary table.
auto_results_tbl <- tibble::tibble(
  model_size = 1:7,
  predictors = vapply(
    1:7,
    function(i) collapse_terms(names(stats::coef(auto_fit_train, id = i))[-1]),
    character(1)
  ),
  test_mse = round(auto_test_mse, 4)
)

# Refit best subset on full data using chosen model size.
auto_best_size <- which.min(auto_test_mse)
auto_fit_full  <- leaps::regsubsets(mpg ~ . - name, data = auto_df, nvmax = 7)
auto_best_coef <- stats::coef(auto_fit_full, id = auto_best_size)

# Print table.
auto_results_tbl %>%
  table_latex(
    col_names  = c('Model Size', 'Predictors', 'Test MSE'),
```

```
    caption    = 'Validation-set comparison for best subset models on Auto.',
    align      = c('c', 'l', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 1: Validation-set comparison for best subset models on Auto.

Model Size	Predictors	Test MSE
1	weight	17.6675
2	weight, year	10.9116
3	weight, year, origin	11.4270
4	weight, acceleration, year, origin	11.4952
5	displacement, weight, acceleration, year, origin	11.3626
6	cylinders, displacement, weight, acceleration, year, origin	11.3731
7	cylinders, displacement, horsepower, weight, acceleration, year, origin	11.3290

Using `mpg ~ . - name`, the candidate variables are `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `year`, and `origin`. With `set.seed(1)`, the split produced 196 training observations and 196 test observations. The lowest test-set MSE is $\boxed{10.9116}$, attained by the 2-variable model with `weight` and `year`. The 1-variable model using only `weight` has test MSE 17.6675, adding `year` lowers the error substantially, and allowing more predictors to enter the larger models does not improve the test MSE on this split. Following the textbook, I then refit best subset selection on the full `Auto` data using the selected model size 2. The resulting final model is

$$\widehat{\texttt{mpg}} = -14.3473 - 0.006632\,\texttt{weight} + 0.7573\,\texttt{year}.$$

Therefore, my final selected model for Problem 1 is the 2-variable best subset model with `weight` and `year`.

Problem 2

Problem 2

Using the `Credit` data from `ISLR2`, build a model to predict `Balance`.

Cell 2

```
# Load data.
credit_df <- ISLR2::Credit
```

Problem 2 (a)

Problem 2 (a)

Consider the best subset method. What are the models that we would come up with considering models with $1, 2, \ldots, 10$ variables? List the results in a table showing the predictors for each model, the R^2 for each model, and the RSS for each model.

Cell 3

```r
# Fit best subset models on full data.
credit_best_subset <- leaps::regsubsets(Balance ~ ., data = credit_df, nvmax = 10)
credit_summary     <- summary(credit_best_subset)

# Create best subset table.
credit_subset_tbl <- tibble::tibble(
  model_size = 1:10,
  predictors = vapply(
    1:10,
    function(i) collapse_terms(names(stats::coef(credit_best_subset, id = i))[-1]),
    character(1)
  ),
  r_squared = round(credit_summary$rsq, 4),
  rss       = round(credit_summary$rss, 2)
)

# Print best subset table.
credit_subset_tbl %>%
  table_latex(
    col_names  = c('Model Size', 'Predictors', '$R^2$', 'RSS'),
    caption    = 'Best subset models for Credit using 1 through 10 variables.',
    align      = c('c', 'l', 'c', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 2: Best subset models for Credit using 1 through 10 variables.

Model Size	Predictors	R^2	RSS
1	Rating	0.7458	21435122
2	Income, Rating	0.8751	10532541
3	Income, Rating, StudentYes	0.9499	4227219
4	Income, Limit, Cards, StudentYes	0.9536	3915058
5	Income, Limit, Rating, Cards, StudentYes	0.9542	3866091
6	Income, Limit, Rating, Cards, Age, StudentYes	0.9547	3821620
7	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes	0.9548	3810759
8	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, RegionWest	0.9549	3804746
9	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes, RegionWest	0.9550	3798367
10	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes, RegionSouth, RegionWest	0.9550	3791345

The best subset results show the best model at each size from 1 through 10. For example, the best 1-variable model uses only `Rating`, the best 3-variable model uses `Income`, `Rating`, and `StudentYes`, and the best 10-variable model uses `Income`, `Limit`, `Rating`, `Cards`, `Age`, `OwnYes`, `StudentYes`, `MarriedYes`, `RegionSouth`, and `RegionWest`.

Problem 2 (b)

Problem 2 (b)

What do you see for R^2 and RSS as you increase the number of predictors? Would you use R^2 or RSS to choose the optimal model from part (a)? Please justify.

As the number of predictors increases, R^2 increases monotonically from 0.7458 for the 1-variable model to 0.9550 for the 10-variable model, while RSS decreases monotonically from 21,435,122 to 3,791,345. This is exactly what we expect: once more predictors are added, the training fit can only improve or remain unchanged.

- I would not use plain R^2 to choose the optimal model, because it is not "fair" to compare models with different numbers of predictors using ordinary R^2. As we increase model size, we lose more degrees of freedom, but ordinary R^2 does not penalize that at all, so it will always favor larger models. If we want an indirect criterion based on R^2, we should instead use adjusted R^2, which explicitly accounts for model size.
- Similarly, I would not use the training RSS by itself as the final selection criterion, because it also always favors larger models. RSS is more useful in the direct approach when it is computed on validation or test data, where it reflects out-of-sample prediction error rather than just in-sample fit.

All in all, neither ordinary R^2 nor training RSS is appropriate for choosing the optimal model; an indirect criterion such as adjusted R^2, AIC, AICc, or BIC would be more appropriate, or a direct criterion such as validation-set or cross-validation error.

Problem 2 (c)

Problem 2 (c)

Comparing AIC measurements of the best models in part (a), what is your optimal model?

Cell 4

```r
# Build AIC table from best subset fits.
credit_matrix   <- stats::model.matrix(Balance ~ ., data = credit_df)
credit_response <- credit_df$Balance
credit_aic      <- sapply(1:10, function(i) {
  xvars  <- names(stats::coef(credit_best_subset, id = i))[-1]
  fit_df <- data.frame(Balance = credit_response, credit_matrix[, xvars, drop = FALSE])
  stats::AIC(stats::lm(Balance ~ ., data = fit_df))
})
credit_aic_tbl <- tibble::tibble(
  model_size = 1:10,
  predictors = credit_subset_tbl$predictors,
  aic        = round(credit_aic, 3)
)

# Print AIC table.
credit_aic_tbl %>%
  table_latex(
    col_names  = c('Model Size', 'Predictors', 'AIC'),
    caption    = 'AIC comparison for the best subset models on Credit.',
    align      = c('c', 'l', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 3: AIC comparison for the best subset models on Credit.

Model Size	Predictors	AIC
1	Rating	5496.782
2	Income, Rating	5214.557
3	Income, Rating, StudentYes	4851.387
4	Income, Limit, Cards, StudentYes	4822.701
5	Income, Limit, Rating, Cards, StudentYes	4819.667
6	Income, Limit, Rating, Cards, Age, StudentYes	4817.039
7	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes	4817.901
8	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, RegionWest	4819.269
9	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes, RegionWest	4820.598
10	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes, RegionSouth, RegionWest	4821.858

The AIC values decrease sharply at first, reach their minimum at the 6-variable model, and then begin to increase again. The minimum AIC is $\boxed{4817.039}$, so the AIC-optimal model is the 6-variable best subset model with `Income`, `Limit`, `Rating`, `Cards`, `Age`, and `StudentYes`. Using the coefficient estimates from this model, the fitted equation is

$$\begin{aligned}\widehat{\texttt{Balance}} = &-493.7342 - 7.7951\,\texttt{Income} + 0.1937\,\texttt{Limit}\\ &+ 1.0912\,\texttt{Rating} + 18.2119\,\texttt{Cards} - 0.6241\,\texttt{Age}\\ &+ 425.6099\,\mathrm{I}\{\texttt{Student} = \texttt{Yes}\}.\end{aligned}$$

Problem 2 (d)

Problem 2 (d)

Use forward variable selection to choose your optimal model and report it.

Cell 5

```r
# Fit forward selection model.
credit_full_lm <- stats::lm(Balance ~ ., data = credit_df)
credit_null_lm <- stats::lm(Balance ~ 1, data = credit_df)
credit_forward <- stats::step(
  credit_null_lm,
  scope     = list(lower = credit_null_lm, upper = credit_full_lm),
  direction = 'forward',
  trace     = 0
)

# Create forward selection table.
credit_forward_tbl <- tibble::tibble(
  method     = 'Forward Selection',
  predictors = collapse_terms(attr(stats::terms(credit_forward), 'term.labels')),
  aic        = round(stats::AIC(credit_forward), 3)
)

# Print forward selection result.
credit_forward_tbl %>%
  table_latex(
    col_names  = c('Method', 'Selected Predictors', 'AIC'),
    caption    = 'Forward Selection Result for Credit.',
    align      = c('c', 'l', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 4: Forward Selection Result for Credit.

Method	Selected Predictors	AIC
Forward Selection	Rating, Income, Student, Limit, Cards, Age	4817.039

Forward Selection chooses the model with `Rating`, `Income`, `Student`, `Limit`, `Cards`, and `Age`, with AIC $\boxed{4817.039}$. So the Forward Selection optimal model is, again, the 6-variable model with `Income`, `Limit`, `Rating`, `Cards`, `Age`, and `Student`.

Problem 2 (e)

Problem 2 (e)

Use backward variable selection to choose your optimal model and report it.

Cell 6

```r
# Fit backward selection model.
credit_backward <- stats::step(
  credit_full_lm,
  direction = 'backward',
  trace     = 0
)

# Create backward selection table.
credit_backward_tbl <- tibble::tibble(
  method     = 'Backward Selection',
  predictors = collapse_terms(attr(stats::terms(credit_backward),
  'term.labels')),
  aic        = round(stats::AIC(credit_backward), 3)
)

# Print backward selection result.
credit_backward_tbl %>%
  table_latex(
    col_names  = c('Method', 'Selected Predictors', 'AIC'),
    caption    = 'Backward Selection Result for Credit.',
    align      = c('c', 'l', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 5: Backward Selection Result for Credit.

Method	Selected Predictors	AIC
Backward Selection	Income, Limit, Rating, Cards, Age, Student	4817.039

Backward Selection also chooses the model with `Income`, `Limit`, `Rating`, `Cards`, `Age`, and `Student`, and its AIC is again $\boxed{4817.039}$. Therefore, Forward Selection, Backward Selection, and the AIC comparison in part (c) all agree on the same 6-variable model.

Problem 2 (f)

Problem 2 (f)

If we split the data into training and test sets, use the training data to conduct best subset model selection, and then check the test error for each model on the test data, which is your optimal model by comparing the testing errors?

Cell 7

```r
# Split data into training and test sets (again, in similiar manner to the aforementioned section on page 271).
set.seed(1)
credit_train <- sample(c(TRUE, FALSE), nrow(credit_df), replace = TRUE)
credit_test  <- !credit_train

# Fit best subset models on training data.
credit_fit_train <- leaps::regsubsets(
  Balance ~ .,
  data  = credit_df[credit_train, ],
  nvmax = 10
)

# Compute test-set MSE for each model size.
credit_test_mse <- rep(NA_real_, 10)
for (i in 1:10) {
  credit_pred        <- predict(credit_fit_train, credit_df[credit_test, ], id = i)
  credit_test_mse[i] <- mean((credit_df$Balance[credit_test] - credit_pred)^2)
}

# Create test-set comparison table.
credit_test_tbl <- tibble::tibble(
  model_size = 1:10,
  predictors = vapply(
    1:10,
    function(i) collapse_terms(names(stats::coef(credit_fit_train, id = i))[-1]),
    character(1)
  ),
  test_mse = round(credit_test_mse, 3)
)
credit_test_best_size <- which.min(credit_test_mse)
credit_full_refit     <- leaps::regsubsets(Balance ~ ., data = credit_df, nvmax = 10)
credit_full_best_coef <- stats::coef(credit_full_refit, id = credit_test_best_size)

# Print test-set comparison table.
credit_test_tbl %>%
  table_latex(
    col_names  = c('Model Size', 'Predictors', 'Test MSE'),
    caption    = 'Validation-set comparison for best subset models on Credit.',
    align      = c('c', 'l', 'c'),
```

```
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 6: Validation-set comparison for best subset models on Credit.

Model Size	Predictors	Test MSE
1	Rating	53192.555
2	Income, Rating	30760.192
3	Income, Rating, StudentYes	10614.653
4	Income, Limit, Cards, StudentYes	9955.349
5	Income, Limit, Cards, OwnYes, StudentYes	10061.285
6	Income, Limit, Rating, Cards, OwnYes, StudentYes	9909.393
7	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes	9779.751
8	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes	9789.864
9	Income, Limit, Rating, Cards, Age, Education, OwnYes, StudentYes, MarriedYes	9786.823
10	Income, Limit, Rating, Cards, Age, OwnYes, StudentYes, MarriedYes, RegionSouth, RegionWest	9811.449

Using `set.seed(1)`, the split gives 201 training observations and 199 test observations. The smallest test-set MSE is $\boxed{9779.751}$, achieved by the 7-variable model. On the training split, that model uses `Income`, `Limit`, `Rating`, `Cards`, `Age`, `OwnYes`, and `StudentYes`. The neighboring models have slightly larger errors: the 6-variable model has test MSE 9909.393, and the 8-variable model has test MSE 9789.864, so the 7-variable model is the best by direct test-set comparison. I then refit best subset selection on the full `Credit` data using the chosen size 7. The full-data best 7-variable model again uses `Income`, `Limit`, `Rating`, `Cards`, `Age`, `OwnYes`, and `StudentYes`, with fitted equation

$$\begin{aligned}\widehat{\texttt{Balance}} = &-488.6159 - 7.8036\,\texttt{Income} + 0.1936\,\texttt{Limit}\\ &+ 1.0940\,\texttt{Rating} + 18.1092\,\texttt{Cards} - 0.6207\,\texttt{Age}\\ &- 10.4532\,\mathrm{I}\{\texttt{Own} = \texttt{Yes}\} + 426.5813\,\mathrm{I}\{\texttt{Student} = \texttt{Yes}\}.\end{aligned}$$

Problem 3

Problem 3

Answer the following conceptual questions about shrinkage methods.

Problem 3 (a)

Problem 3 (a)

From your learning of the shrinkage methods, why must we standardize the data?

We standardize the predictors so that the penalty treats all coefficients on the same scale. Without standardization, a predictor measured in large units could have a very small coefficient and therefore be penalized less, while a predictor measured in small units could have a larger coefficient and be penalized more, even if the two predictors are equally important. Ridge and lasso penalize coefficient size, so if the predictors are not standardized, the penalty depends partly on the units of measurement rather than only on predictive contribution.

Problem 3 (b)

Problem 3 (b)

How do we write the model formula if we build a model using ridge or lasso?

The linear model itself is written in the usual form

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon.$$

What changes is the criterion used to estimate the coefficients. Ridge regression minimizes

$$\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2,$$

while lasso minimizes

$$\sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j|.$$

So the model formula is still linear in the predictors, but the fitting procedure adds either an L_2 penalty (ridge) or an L_1 penalty (lasso).

Problem 3 (c)

Problem 3 (c)

Can ‘glmnet‘ be used for generalized models, such as logistic regression?

Yes. `glmnet` can be used for generalized linear models, including logistic regression, by specifying the appropriate `family` argument. For example, logistic regression is fit with `family = 'binomial'`. So `glmnet` is not limited to ordinary least squares; it can also handle generalized responses while still applying ridge, lasso, or elastic-net regularization. In fact, I actually used this in my new “legacy” back-filling version of my expected-goals (xG) model on Rento’s Rink, for which you can read an interactive article here.